

# **Sviluppo guidato dai test e Refactoring**

Oliviero Riganelli

# Sviluppo guidato dai test

Sviluppo guidato dal test – test-driven development (TDD)

- una best practice promossa dal metodo agile e iterativo XP (Applicabile anche a UP e Scrum)
- noto anche come sviluppo preceduto dai test
- consideriamo qui solo il test unitario (ovvero i test relativi a singole classi e metodi)

# Idea di base del TDD

- il test è scritto prima del codice da testare
  - *immaginando che il codice da testare sia stato già scritto*
- poi si scrive il codice per far passare il test

# Vantaggi del TDD

- I test vengono effettivamente scritti
- La soddisfazione del programmatore porta ad una scrittura di test più coerente
- Chiarezza dell'interfaccia e del comportamento dettagliati
- Verifica automatica, ripetibile e dimostrabile
- Fiducia nel cambiare le cose

# Tipi di test

Test di unità



Test di integrazione



Test di sistema



Test di accettazione



# Schema dei metodi di test di unità

- **Preparazione:** crea l'oggetto da verificare (detto anche fixture)
- **Esecuzione:** fare qualcosa alla fixture al fine di eseguire alcune operazioni che si desidera testare
- **Verifica:** verifica che i risultati ottenuti corrispondono a quelli previsti
- **Rilascio:** opzionalmente rilascia o ripulisce gli oggetti e le risorse utilizzate nel test

# Utilizzare xUnit

- xUnit è un framework per test di unità per molti linguaggi, es:
  - *NUnit per .NET*
  - *JUnit per java*

# Esempio sistema Pos NextGen

Per testare la classe Sale, bisogna creare una classe SaleTest che:

- Crea un oggetto Sale (**preparazione**)
- Aggiunge alla Sale alcuni elementi di riga con il metodo makeLineItem, che è il comportamento che si vuole verificare (**esecuzione**)
- Richiede il totale e ne verifica il valore atteso (**verifica**)



# Esempio sistema Pos NextGen

```
import static org.junit.Assert.*;
import org.junit.*;

import com.foo.nextgen.domain.*;

public class SaleTest {

    @Test
    // test per il metodo Sale.makeLineItem
    public void testMakeLineItem() {
        // PREPARAZIONE ←
        // - crea la fixture, ovvero l'oggetto da testare
        // - un possibile idioma è chiamarlo 'fixture'
        // - spesso è definito come una variabile d'istanza
        //   anziché come una variabile locale
        Sale sale = new Sale();
        // - crea degli oggetti di supporto per il test
        ProductDescription tofu =
            new ProductDescription( new ItemID( "1" ),
                                   new Money( 2.50 ),
                                   "Tofu" );
        ProductDescription burger =
            new ProductDescription( new ItemID( "2" ),
                                   new Money( 1.50 ),
                                   "VeggieBurger" );

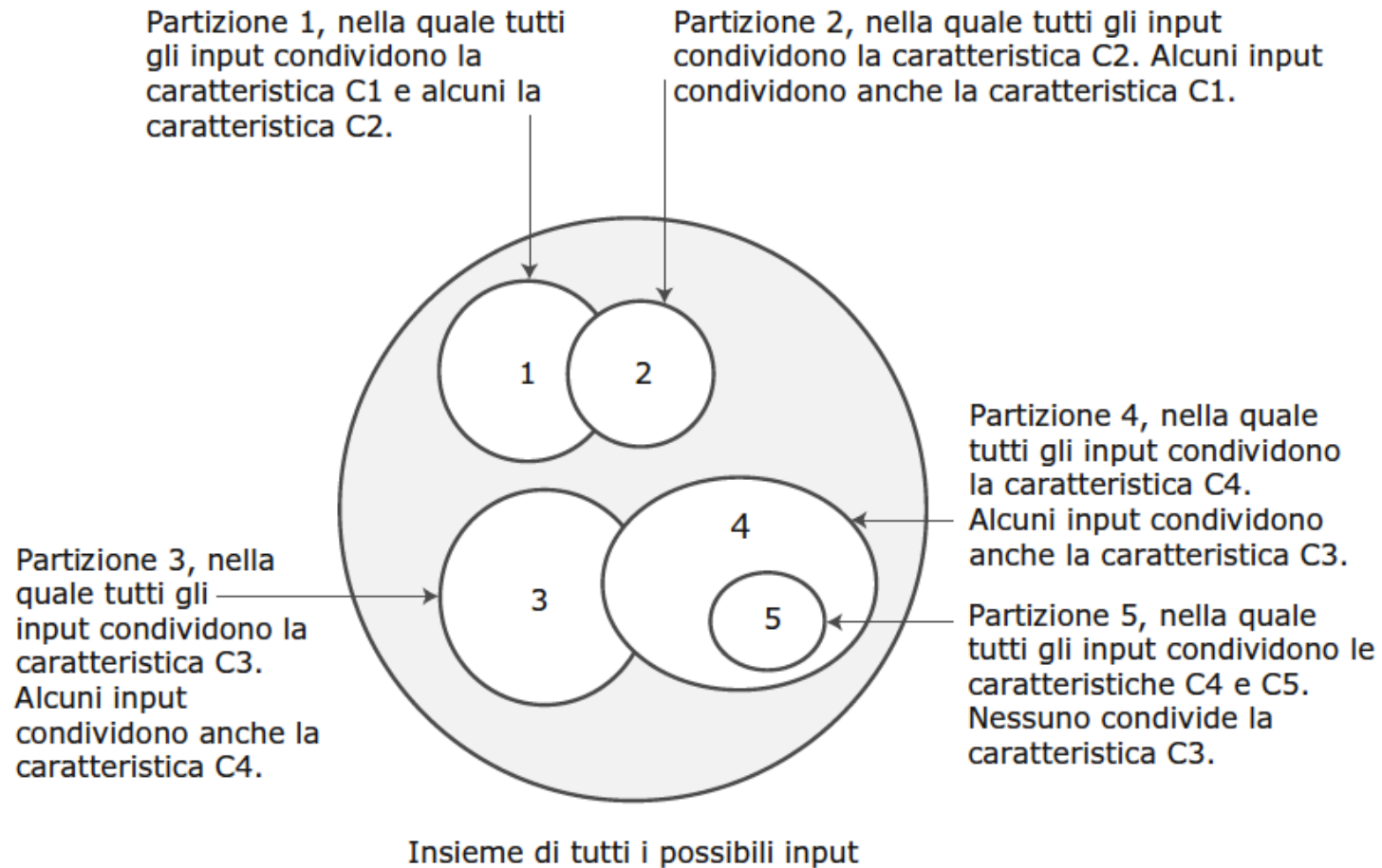
        // ESECUZIONE ←
        // questo codice viene scritto **immaginando** che
        // ci sia già un metodo makeLineItem.
        // Questo atto di immaginazione mentre viene scritto
        // il test tende a migliorare o a chiarire la nostra
        // comprensione dell'interfaccia dettagliata dell'oggetto.
        // Pertanto il TDD ha il vantaggio collaterale di
        // chiarire la progettazione a oggetti dettagliata.
        sale.makeLineItem( tofu, 2 );
        sale.makeLineItem( burger, 1 );

        // VERIFICA ←
        // confronta il risultato atteso con quello effettivo
        assertEquals( new Money(6.50), sale.getTotal() );
    }
}
```

# Input

- Per testare un programma in modo efficiente è necessario identificare insiemi di input che siano trattati allo stesso modo nel codice
  - Tali insiemi sono detti *partizioni di equivalenza*
- Le partizioni identificate non dovrebbero includere solo quelle contenenti input che producono risultati corretti
  - È opportuno identificare anche partizioni di non correttezza nelle quali gli input siano deliberatamente sbagliati
  - Serviranno a testare che il programma rilevi e gestisca nel modo atteso input non corretti

# Partizioni di equivalenza



# Input del test

- Il programma dovrebbe essere testato con svariati input da ciascuna partizione di equivalenza
- Se possibile è bene edificare i limiti della partizione e scegliere input vicini a tali limiti
- Ampie esperienze di testing hanno mostrato che i programmatori commettono errori con i valori ai limiti (i valori di frontiera)

# Esempio: Funzione Verifica Nomi

```
def controllonome (s):  
  
    # controlla che un nome contenga solo caratteri alfabetici, - o apici singoli  
    # i nomi devono essere lunghi da 2 a 40 caratteri  
    # non sono consentite stringhe tra apici e --  
  
    namex = r"^[a-zA-Z][a-zA-Z-']{1,39}$"  
    if re.match (namex, s):  
        if re.search ("'.*'", s) or re.search ("--", s):  
            return False  
        else:  
            return True  
    else:  
        return False
```

---

# Esempio: regole per l'input

- il parametro di input (il nome di una persona) rispetta le seguenti regole:
  - Nome deve essere lungo da 2 a 40 caratteri
  - I caratteri del nome devono essere alfabetici o alfabetici accentati, più piccolo numero di caratteri separatori speciali
  - Gli unici caratteri separatori non alfabetici consistenti sono trattino e apostrofo. I nomi devono iniziare con una lettera

# Esempio: Partizioni di equivalenza per la funzione verifica nomi

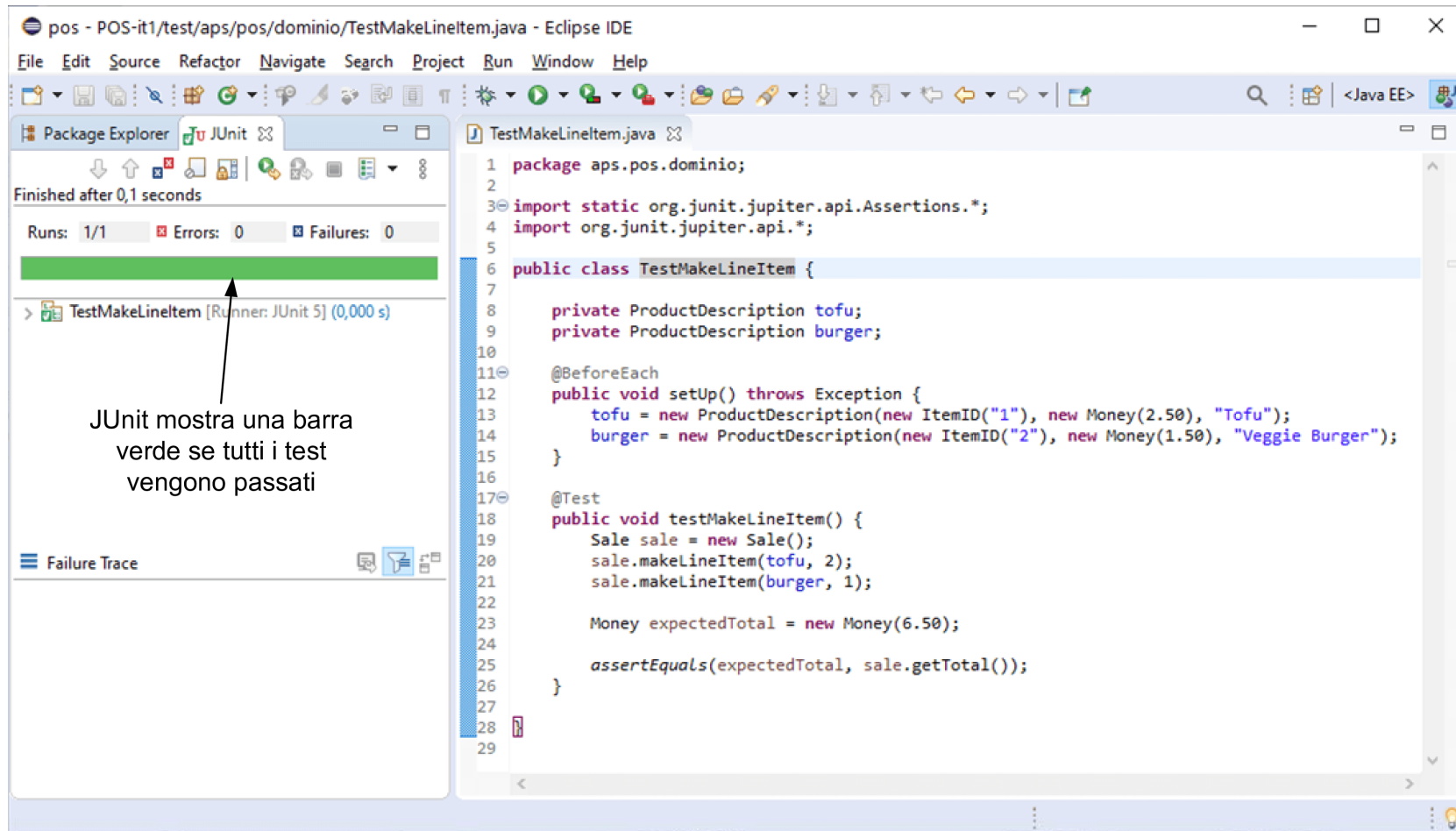
| Partizione di equivalenza | Caratteristica                                                                                                                         |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Nomi corretti 1           | Gli input includono solo caratteri alfabetici e sono lunghi da 2 a 40 caratteri.                                                       |
| Nomi corretti 2           | Gli input includono solo caratteri alfabetici, trattini o apostrofi e sono lunghi da 2 a 40 caratteri.                                 |
| Nomi non corretti 1       | Gli input sono lunghi da 2 a 40 caratteri ma contengono caratteri non consentiti.                                                      |
| Nomi non corretti 2       | Gli input includono solo caratteri consentiti, ma sono lunghi solo 1 carattere o più di 40.                                            |
| Nomi non corretti 3       | Gli input sono lunghi da 2 a 40 caratteri, ma il primo carattere è un trattino o un apostrofo.                                         |
| Nomi non corretti 4       | Gli input contengono solo caratteri validi e sono lunghi da 2 a 40 caratteri, ma includono doppi trattini, testo tra apici o entrambi. |

# Linee guida per il testing di unità

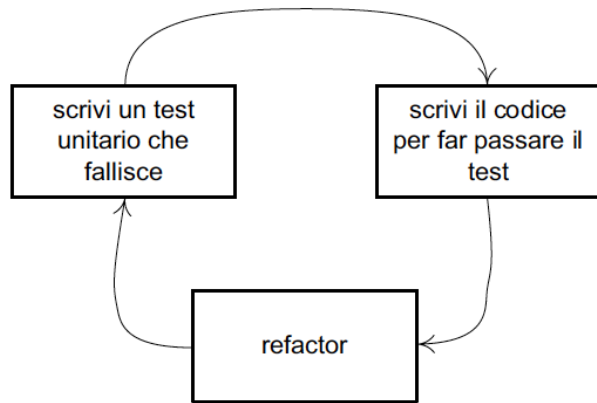
| Linea guida                        | Spiegazione                                                                                                                                                                                       |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Testare casi limite (di frontiera) | Se la partizione ha dei limiti minimi e massimi (come la lunghezza di stringhe, numeri e così via), si scelgano input ai limiti dell'intervallo.                                                  |
| Forzare errori                     | Si scelgano input che forzino il sistema a generare tutti i messaggi d'errore. Si scelgano input che dovrebbero generare output non validi.                                                       |
| Riempire i buffer                  | Si scelgano input che causino l'overflow di tutti i buffer di input.                                                                                                                              |
| Ripetersi                          | Si ripeta più volte lo stesso input o la stessa serie di input.                                                                                                                                   |
| Overflow e underflow               | Se il programma esegue calcoli numerici, si scelgano input che causano il calcolo di numeri molto grandi o molto piccoli.                                                                         |
| Non dimenticare null e zero        | Se il programma usa puntatori o stringhe, si testi sempre con puntatori e stringhe null. Se si usano sequenze, si testi con una sequenza vuota. Per gli input numerici, si testi sempre con zero. |
| Tenere il conto                    | In presenza di liste e trasformazioni di liste, si tenga conto del numero di elementi in ogni lista e si verifichi che siano coerenti dopo ogni trasformazione.                                   |
| Uno è diverso                      | Se il programma opera con sequenze, si testi sempre con sequenze che hanno un solo valore.                                                                                                        |



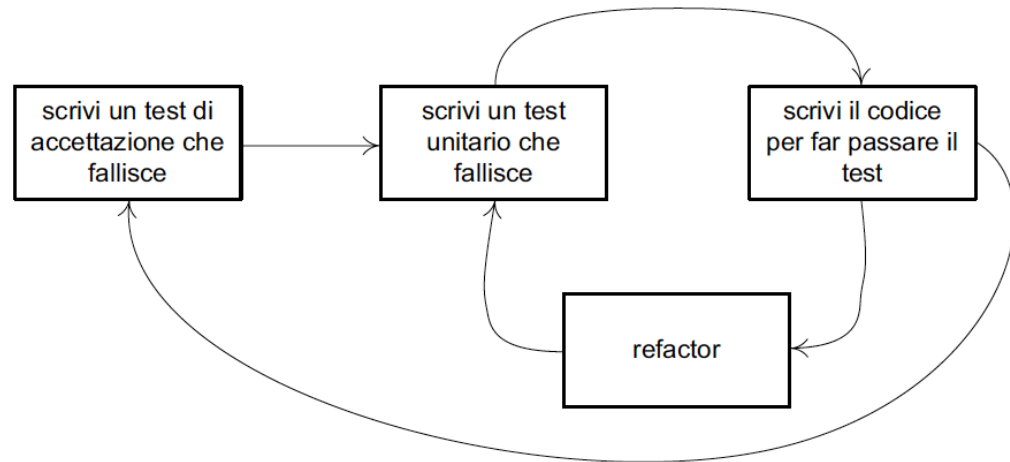
# Supporto degli IDE al TDD



# Cicli del TDD



Il ciclo di base del TDD per i test unitari.



Il doppio ciclo del TDD.

# Linee guida

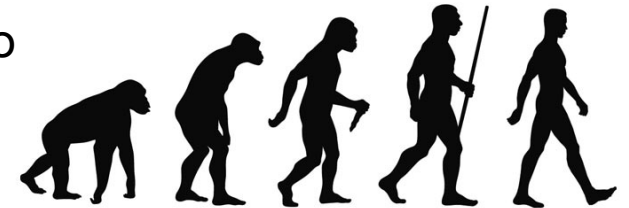
- non scrivere codice di produzione fino a quando non hai scritto un test unitario che fallisce
- non scrivere più di un test unitario di quanto è sufficiente a far fallire la compilazione o l'esecuzione del test
- non scrivere più codice di produzione di quanto è sufficiente a passare il test che fallisce

**le regole sono semplici – ma la loro applicazione non è per niente banale!**

# Refactoring

Il Refactoring è

- un metodo strutturato e disciplinato
- per riscrivere o ristrutturare del codice esistente
- senza modificare il suo comportamento esterno
- applicando piccoli passi di trasformazione
- in combinazione con la ripetizione dei test dopo ciascun passo



---

## Refactoring

Migliorare la progettazione di un codice esistente

# Refactoring

- Che utilità può avere un refactoring
  - *Miglioramento continuo del codice*
  - *Preparazione al cambiamento*
- Obiettivi e attività di una buona programmazione
  - *rimuovere codice duplicato*
  - *Migliorare la chiarezza*
  - *Abbreviare metodi lunghi*
  - *E altro ...*

# Quando?

- Regola del Tre
- Quando si aggiunge la funzionalità
- Quando si corregge un bug
- Durante la revisione del codice

# Passi

- Assicuratevi che i vostri test passino
- Trovate un code smell
- Determinate come migliorare questo codice
- Effettuate le migliorie
- Eseguite i test per assicurarsi che le cose funzionino ancora correttamente
- Ripetete il ciclo di miglioramento/test fino a quando il code smell non è sparito

# Code smell



## Bloaters

- Long Method
- Large Class
- Primitive Obsession
- Long Parameter List
- Data Clumps



## Change Preventer

- Divergent Change
- Shotgun Surgery
- Parallel Inheritance Hierarchies



## Object-Orientation Abuser

- Switch Statements
- Temporary Field
- Refused Bequest
- Alternative Classes with Different Interfaces



## Coupler

- Feature Envy
- Inappropriate Intimacy
- Message Chains
- Middle Man
- Incomplete Library Class



## Dispensable

- Comments
- Duplicate Code
- Lazy Class
- Data Class
- Dead Code
- Speculative Generality



# Gli smell suggeriscono il refactoring

## Dealing with Generalisation

- 
- Pull Up Field
  - Pull Up Method
  - Pull Up Constructor Body
  - Push Down Method
  - Push Down Field
  - Extract Subclass
  - Extract Superclass
  - Extract Interface
  - Collapse Hierarchy
  - Form Template Method
  - Replace Inheritance with Delegation
  - Replace Delegation with Inheritance

## Moving Features between Object

- 
- Move Method
  - Move Field
  - Extract Class
  - Inline Class
  - Hide Delegate
  - Remove Middle Man
  - Introduce Foreign Method
  - Introduce Local Extension

## Organizing Data

- 
- Self Encapsulate Field
  - Replace Data Value with Object
  - Change Value to Reference
  - Change Reference to Value
  - Replace Array with Object
  - Duplicate Observed Data
  - Change Unidirectional Association to Bidirectional
  - Change Bidirectional Association to Unidirectional
  - Replace Magic Number with Symbolic Constant
  - Encapsulate Field
  - Encapsulate Collection
  - Replace Type Code with Class
  - Replace Type Code with Subclasses
  - Replace Type Code with State/Strategy
  - Replace Subclass with Fields

## Simplifying Conditional Expressions

- 
- Decompose Conditional
  - Consolidate Conditional Expression
  - Consolidate Duplicate Conditional Fragments
  - Remove Control Flag
  - Replace Nested Conditional with Guard Clauses
  - Replace Conditional with Polymorphism
  - Introduce Null Object
  - Introduce Assertion

## Simplifying Method Calls

- 
- Rename Method
  - Add Parameter
  - Remove Parameter
  - Separate Query from Modifier
  - Parameterize Method
  - Replace Parameter with Explicit Methods
  - Preserve Whole Object
  - Replace Parameter with Method Call
  - Introduce Parameter Object
  - Remove Setting Method
  - Hide Method
  - Replace Constructor with Factory Method
  - Replace Error Code with Exception
  - Replace Exception with Test

## Composing method

- 
- Extract Method
  - Inline Method
  - Extract Variable
  - Inline Temp
  - Replace Temp with Query
  - Split Temporary Variable
  - Remove Assignments to Parameters
  - Replace Method with Method Object
  - Substitute Algorithm

# Long Method

- Sintomo: Un metodo contiene troppe righe di codice (più lungo di dieci righe)
- Problema: i metodi lunghi sono difficili da capire, modificare, riutilizzare
- Soluzione:
  - *Per ridurre la lunghezza del corpo del metodo, utilizzare il **Extract Method**.*
  - *Se le variabili e i parametri locali interferiscono con l'estrazione di un metodo, usare **Replace Temp with Query**, **Introduce Parameter Object** o **Preserve Whole Object**.*
  - *Se nessuna delle ricette precedenti aiuta, provare a spostare l'intero metodo in un oggetto separato tramite **Replace Method with Method Object**.*
  - *i loop e gli operatori condizionali sono un buon segno che il codice può essere spostato in un metodo separato. Per i condizionamenti, usare **Decompose Conditional**. Per i loop, provare **Extract Method**.*

# Extract Method

Il metodo *takeTurn* prima del refactoring.

```
public class Player {  
  
    private Piece piece;  
    private Board board;  
    private Die[] dice;  
    // ...  
  
    public void takeTurn() {  
        int rollTotal = 0;  
        for (int i=0; i<dice.length; i++) {  
            dice[i].roll();  
            rollTotal += dice[i].getFaceValue();  
        }  
        Square oldLoc = piece.getLocation();  
        Square newLoc = board.getSquare(oldLoc, rollTotal);  
        piece.setLocation(newLoc);  
    }  
}
```

# Extract Method

Il codice dopo il refactoring con *Extract Method*.

```
// metodo di supporto estratto dal refactoring
private int rollDice() {
    int rollTotal = 0;
    for (int i=0; i<dice.length; i++) {
        dice[i].roll();
        rollTotal += dice[i].getFaceValue();
    }
    return rollTotal;
}
}
```



# Extract Method

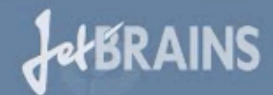
IntelliJ IDEA Basics

Breandan Considine

*JetBrains Developer Advocate*

---

© JetBrains. All rights reserved



# Extract Method

```
public Player(String name, Die[] dice, Board board)
{
    this.name = name;
    this.dice = dice;
    this.board = board;
    piece = new Piece(board.getStartSquare());
}

public void takeTurn()
{
    // roll dice
    int rollTotal = 0;
    for (int i = 0; i < dice.length; i++)
    {
        dice[i].roll();
        rollTotal += dice[i].getFaceValue();
    }

    Square newLoc = board.getSquare(piece.getLocation(), rollTotal);
    piece.setLocation(newLoc);
}
```

Extract Method

Method name:

Access modifier: ☐ public ☐ protected ☐ default ☒ private

☐ Add thrown runtime exceptions to method signature

☐ Generate Javadoc comment

☐ Replace duplicate code fragments

Method signature preview:

**private int rollDice()**

Preview > OK Cancel

```
public void takeTurn()
{
    int rollTotal = rollDice();

    Square newLoc = board.getSquare(piece.getLocation(), rollTotal);
    piece.setLocation(newLoc);
}

private int rollDice()
{
    // roll dice
    int rollTotal = 0;
    for (int i = 0; i < dice.length; i++)
    {
        dice[i].roll();
        rollTotal += dice[i].getFaceValue();
    }
    return rollTotal;
}
```

IDE prima del refactoring

IDE dopo il refactoring

# Replace Temp with query

**Il risultato di un'espressione viene inserito in una variabile locale per essere utilizzato successivamente nel codice.**

```
const basePrice = this._quantity * this._itemPrice;  
if (basePrice > 1000)  
  return basePrice * 0.95;  
else  
  return basePrice * 0.98;
```



```
get basePrice() {this._quantity * this._itemPrice;}  
  
...  
  
if (this.basePrice > 1000)  
  return this.basePrice * 0.95;  
else  
  return this.basePrice * 0.98;
```

**Spostare l'intera espressione in un metodo separato e restituirne il risultato. Interrogare il metodo invece di usare una variabile.**

# Introduce Parameter Object

**I metodi contengono un gruppo ripetuto di parametri.**

```
function amountInvoiced(startDate, endDate) {...}  
function amountReceived(startDate, endDate) {...}  
function amountOverdue(startDate, endDate) {...}
```



```
function amountInvoiced(aDateRange) {...}  
function amountReceived(aDateRange) {...}  
function amountOverdue(aDateRange) {...}
```

**Sostituire questi parametri con un oggetto**



# Preserve Whole Object

Si ottengono diversi valori da un oggetto e poi li si passa come parametri a un metodo.

```
const low = aRoom.daysTempRange.low;  
const high = aRoom.daysTempRange.high;  
if (aPlan.withinRange(low, high))
```



```
if (aPlan.withinRange(aRoom.daysTempRange))
```

**Si può invece provare a passare l'intero oggetto.**

# Replace Method with Method Object

Si ha un metodo lungo in cui le variabili locali sono così intrecciate che non è possibile applicare il metodo Extract.

```
function score(candidate, medicalExam, scoringGuide) {  
  let result = 0;  
  let healthLevel = 0;  
  // long body code  
}
```



```
class Scorer {  
  constructor(candidate, medicalExam, scoringGuide) {  
    this._candidate = candidate;  
    this._medicalExam = medicalExam;  
    this._scoringGuide = scoringGuide;  
  }  
  
  execute() {  
    this._result = 0;  
    this._healthLevel = 0;  
    // long body code  
  }  
}
```

**Trasformare il metodo in una classe separata, in modo che le variabili locali diventino campi della classe. Quindi è possibile suddividere il metodo in più metodi all'interno della stessa classe.**

# Decompose Conditional

**Si ha una condizione complessa**

```
if (!aDate.isBefore(plan.summerStart) && !aDate.isAfter(plan.summerEnd))  
    charge = quantity * plan.summerRate;  
else  
    charge = quantity * plan.regularRate + plan.regularServiceCharge;
```



```
if (summer())  
    charge = summerCharge();  
else  
    charge = regularCharge();
```

**Decomporre le parti complicate della condizione in metodi separati**

# Duplicated Code

- Sintomo: Lo stesso codice o molto simile appare in molti luoghi
- Problema: Un fix in un codice duplicato clone non può essere propagata a tutti. Rende il codice più grande di quello che deve essere
- Soluzione:
  - *Se lo stesso codice si trova in due o più metodi nella stessa classe: usare il Extract Method e posizionare le chiamate per il nuovo metodo in entrambi i posti.*
  - *Per altre soluzioni <https://sourcemaking.com/refactoring/smells/duplicate-code>*

# Example

```
def findAManager:
  for each employee in employees:
    if employee.isManager:
      return employee
  return nil
```

```
def findASupervisor:
  for each employee in employees:
    if employee.isSupervisor:
      return employee
  return nil
```

```
def findAWorkerBee:
  for each employee in employees:
    if employee.isWorkerBee:
      return employee
  return nil
```

# Example

```
def findAManager:
  for each employee in employees:
    if employee.isManager:
      return employee
  return nil
```

```
def findASupervisor:
  for each employee in employees:
    if employee.isSupervisor:
      return employee
  return nil
```

```
def findAWorkerBee:
  for each employee in employees:
    if employee.isWorkerBee:
      return employee
  return nil
```



```
def findType(type)
  for each employee in employees:
    if employee.isType(type)
      return employee
  return nil
```

```
def findAManager:
  return findType (lambda worker: worker.isManager)
```

```
def findASupervisor:
  return findType (lambda worker: worker.isSupervisor)
```

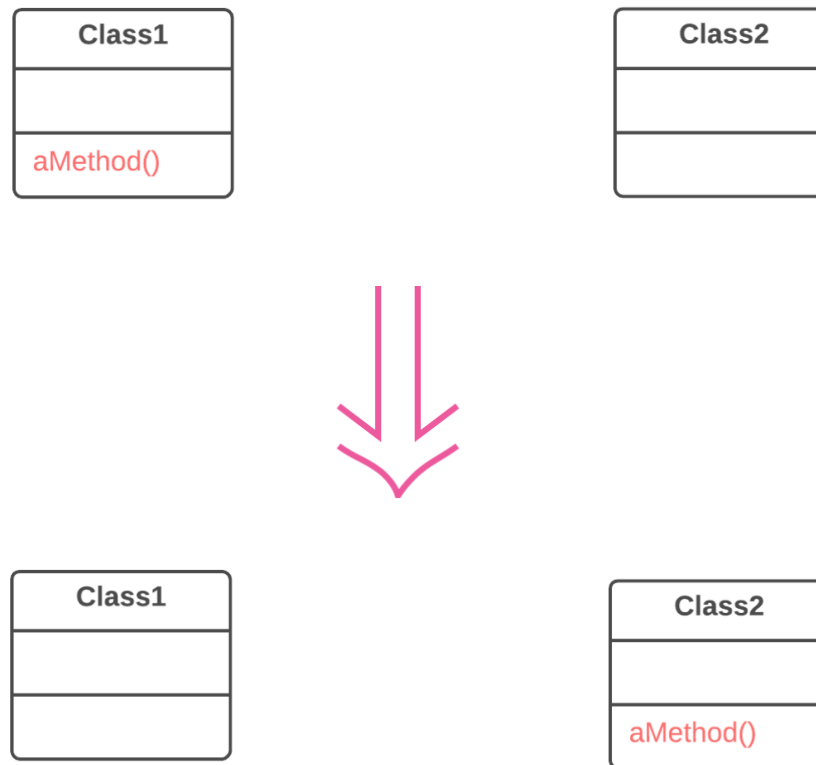
```
def findAWorkerBee:
  return findType (lambda worker: worker.isWorkerBee)
```

# Feature Envy

- Sintomo: Un metodo accede ai dati di un altro oggetto più che ai propri dati.
- Problema: Errore di progettazione, metodo erroneamente inserito nella classe sbagliata
- Soluzione:
  - *Se un metodo deve essere chiaramente spostato in un altro luogo, utilizzare il **Move Method**.*
  - *Se solo una parte di un metodo accede ai dati di un altro oggetto, usare il **Extract Method***
  - *Se un metodo utilizza funzioni di diverse altre classi, determinare innanzitutto quale classe contiene la maggior parte dei dati utilizzati. Poi collocare il metodo in questa classe insieme agli altri dati. In alternativa, usare **Extract Method** per dividere il metodo in diverse parti che possono essere collocate in diversi posti in diverse classi.*

# Move Method

Un metodo viene utilizzato più in un'altra classe che nella propria classe.



**Creare un nuovo metodo nella classe che utilizza maggiormente il metodo, quindi spostare il codice del vecchio metodo lì. Trasformare il codice del metodo originale in un riferimento al nuovo metodo nell'altra classe, oppure eliminarlo del tutto.**



# Large Class

- Sintomo: Una classe contiene molti campi/metodi/linee di codice.
- Problema: Indica un errore di astrazione, è probabile che ci sia più di un interesse incorporata nel codice o, alcuni metodi appartengono ad altre classi
- Soluzione:
  - **Extract Class** prende un sottoinsieme delle variabili di istanza e dei metodi e crea una nuova classe con essi. Questo rende la classe iniziale (lunga) più breve
  - **Move Method** sposta uno o più metodi in altre classi
  - **Extract Subclass** crea una sottoclasse per una parte de comportamento della classe, se questo è usato solo in alcuni casi

# Extract Class

**Quando una classe fa il lavoro di due, si crea confusione.**

```
class Person {  
    get officeAreaCode() {return this._officeAreaCode;}  
    get officeNumber()   {return this._officeNumber;}  
}
```



```
class Person {  
    get officeAreaCode() {return this._telephoneNumber.areaCode;}  
    get officeNumber()   {return this._telephoneNumber.number;}  
}  
  
class TelephoneNumber {  
    get areaCode() {return this._areaCode;}  
    get number()   {return this._number;}  
}
```

**Invece, si crea una nuova classe e vi si inseriscono i campi e i metodi responsabili della funzionalità in questione.**

# Extract Subclass

Una classe ha caratteristiche che vengono utilizzate solo in determinati casi.



**Creare una sottoclasse e utilizzarla in questi casi**

# Switch Statement

- Sintomo: Si ha un complesso operatore di Switch o di una sequenza di istruzioni If.
- Problema: La stessa struttura Switch/Case appare in molti punti
- Soluzione:
  - *Dopo aver specificato la struttura di eredità, usare **Replace Conditional with Polymorphism***
  - *Per isolare il `switch` e metterlo nella sottoclasse giusta, potrebbe essere necessario **Extract Method** e poi **Move Method**.*

# Replace Conditional with Polymorphism

Si dispone di un condizionale che esegue varie azioni in base al tipo di oggetto o alle proprietà.

```
switch (bird.type) {  
  case 'EuropeanSwallow':  
    return "average";  
  case 'AfricanSwallow':  
    return (bird.numberOfCoconuts > 2) ? "tired" : "average";  
  case 'NorwegianBlueParrot':  
    return (bird.voltage > 100) ? "scorched" : "beautiful";  
  default:  
    return "unknown";  
}
```



```
class EuropeanSwallow {  
  get plumage() {  
    return "average";  
  }  
}  
class AfricanSwallow {  
  get plumage() {  
    return (this.numberOfCoconuts > 2) ? "tired" : "average";  
  }  
}  
class NorwegianBlueParrot {  
  get plumage() {  
    return (this.voltage > 100) ? "scorched" : "beautiful";  
  }  
}
```

**Creare sottoclassi corrispondenti ai rami della condizione. In esse, creare un metodo condiviso e spostare il codice dal ramo corrispondente della condizione ad esso. Quindi sostituire il condizionale con la chiamata al metodo corrispondente. Il risultato è che l'implementazione corretta sarà ottenuta tramite il polimorfismo, a seconda della classe dell'oggetto.**

# Data Class

- Sintomo: Una classe che ha solo variabili di classe, metodi/proprietà getter/setter e nient'altro. Sta solo agendo come contenitore di dati.
- Problema: Tipicamente, altre classi hanno metodi con Feature Envy. Indica che il progetto è veramente procedurale
- Soluzione:
  - *Esaminare i metodi che utilizzano i dati nella data class e usare **Move Method** o **Extract Method** per migrare questa funzionalità nella data class.*

# Long Parameter List

- Sintomo: Molti parametri passati in un metodo.
- Problema: È difficile comprendere tali elenchi, che diventano contraddittori e difficili da usare man mano che si allungano.
- Soluzione:
  - *Se alcuni degli argomenti sono solo risultati di chiamate di metodo di un altro oggetto, usare **Replace Parameter with Method Call**.*
  - *Invece di passare un gruppo di dati ricevuti da un altro oggetto come parametri, passare l'oggetto stesso al metodo, utilizzando **Preserve Whole Object**.*
  - *Se ci sono più elementi dati non correlati tra loro, a volte è possibile unirli in un unico oggetto parametro tramite **Introduce Parameter Object**.*

# Replace Parameter with Method Call

**Chiamare un metodo di query e passare i suoi risultati come parametri di un altro metodo, mentre quest'ultimo potrebbe chiamare direttamente la query.**

```
int basePrice = quantity * itemPrice;  
  
double seasonDiscount = this.getSeasonalDiscount();  
  
double fees = this.getFees();  
  
double finalPrice = discountedPrice(basePrice, seasonDiscount, fees);
```



```
int basePrice = quantity * itemPrice;  
  
double finalPrice = discountedPrice(basePrice);
```

**Invece di passare il valore attraverso un parametro, provare a inserire una chiamata di query all'interno del corpo del metodo.**

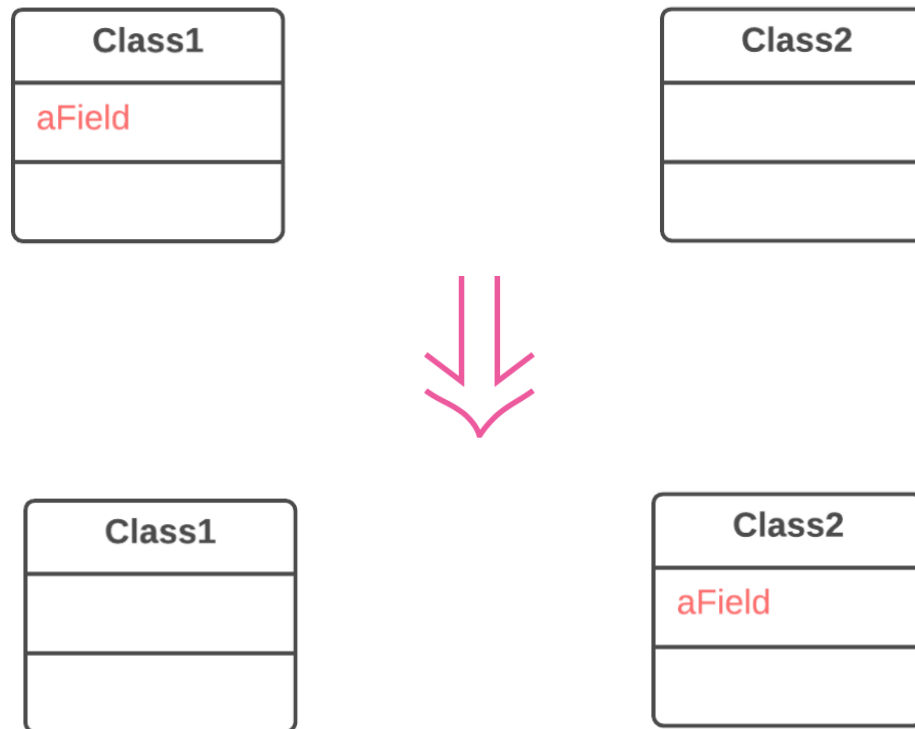


# Shotgun Surgery

- Sintomo: Per apportare eventuali modifiche è necessario apportare molte piccole modifiche a molte classi diverse.
- Problema: Un'unica responsabilità è stata suddivisa in un gran numero di classi.
- Soluzione:
  - Usare il metodo **Move Method** e **Move Field** per spostare i comportamenti di classe esistenti in una singola classe. Se non c'è una classe appropriata per questo, crearne una nuova.
  - Se lo spostamento del codice nella stessa classe lascia le classi originali quasi vuote, provare a sbarazzarsi di queste classi ormai ridondanti tramite *Inline Class*.

# Move Field

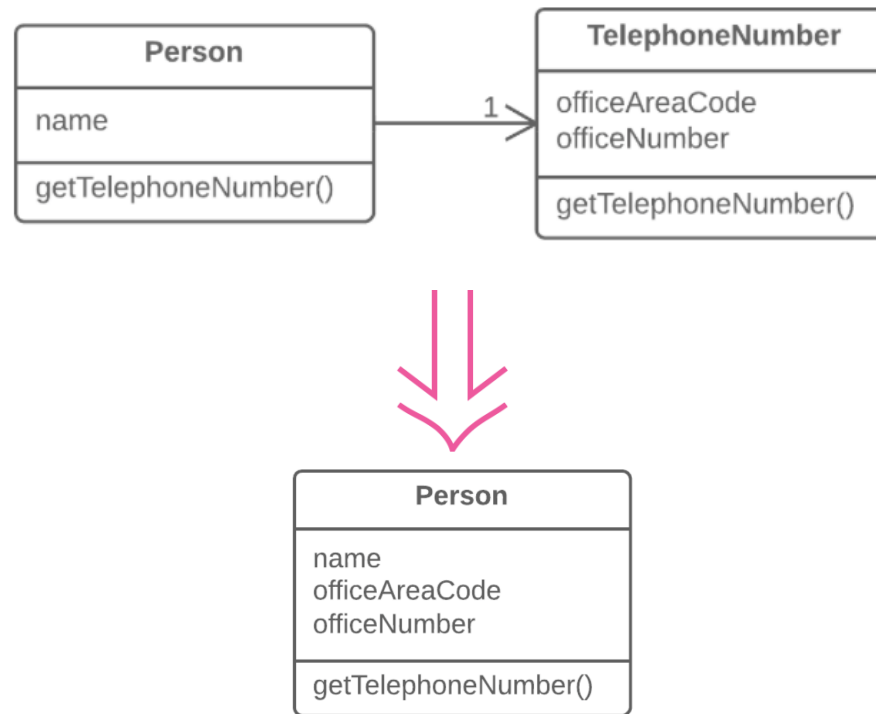
Un campo viene utilizzato più in un'altra classe che nella sua stessa classe.



Creare un campo in una nuova classe e reindirizzare tutti gli utenti del vecchio campo ad esso.

# Inline Class

Una classe non fa quasi nulla, non è responsabile di nulla e non è prevista alcuna responsabilità aggiuntiva.



**Sposta tutte le caratteristiche dalla classe a un'altra.**

# Comment

- Sintomo: Un metodo è ricco di commenti esplicativi.
- Problema: Il codice dovrebbe essere auto-esplicativo, quindi se avete bisogno di lunghi commenti, il vostro codice non è chiaro. Soluzione:
  - *Se un commento è inteso a spiegare un'espressione complessa, l'espressione deve essere suddivisa in sottoespressioni comprensibili utilizzando **Extract Variable**.*
  - *Se un commento spiega una sezione di codice, questa sezione può essere trasformata in un metodo separato tramite **Extract Method**. Il nome del nuovo metodo può essere preso dal testo del commento stesso, molto probabilmente.*
  - *Se un metodo è già stato estratto, ma sono ancora necessari dei commenti per spiegare cosa fa il metodo, date al metodo un nome autoesplicativo. Utilizzare **Rename Method** per questo.*
  - *Se avete bisogno di affermare delle regole su uno stato che è necessario per il funzionamento del sistema, usate **Introduce Assertion**.*

# Extract Variable

**C'è un'espressione che è difficile da capire.**

```
return order.quantity * order.itemPrice -  
    Math.max(0, order.quantity - 500) * order.itemPrice * 0.05 +  
    Math.min(order.quantity * order.itemPrice * 0.1, 100);
```



```
const basePrice = order.quantity * order.itemPrice;  
const quantityDiscount = Math.max(0, order.quantity - 500) * order.itemPrice * 0.05;  
const shipping = Math.min(basePrice * 0.1, 100);  
return basePrice - quantityDiscount + shipping;
```

**Collocare il risultato dell'espressione o le sue parti in variabili separate che siano autoesplicative.**

# Rename Method

**Il nome di un metodo non spiega cosa fa.**

```
function circum(radius) {...}
```



```
function circumference(radius) {...}
```

**Rinominare il metodo**

# Introduce Assertion

**Affinché una porzione di codice funzioni correttamente, alcune condizioni o valori devono essere veri.**

```
double getExpenseLimit() {  
    // Deve avere un limite di spesa o  
    // un progetto primario.  
    return (expenseLimit != NULL_EXPENSE) ?  
        expenseLimit :  
        primaryProject.getMemberExpenseLimit();  
}
```



```
double getExpenseLimit() {  
    Assert.isTrue(expenseLimit != NULL_EXPENSE || primaryProject != null);  
    return (expenseLimit != NULL_EXPENSE) ?  
        expenseLimit:  
        primaryProject.getMemberExpenseLimit();  
}
```

**Sostituire queste ipotesi con verifiche specifiche delle asserzioni.**

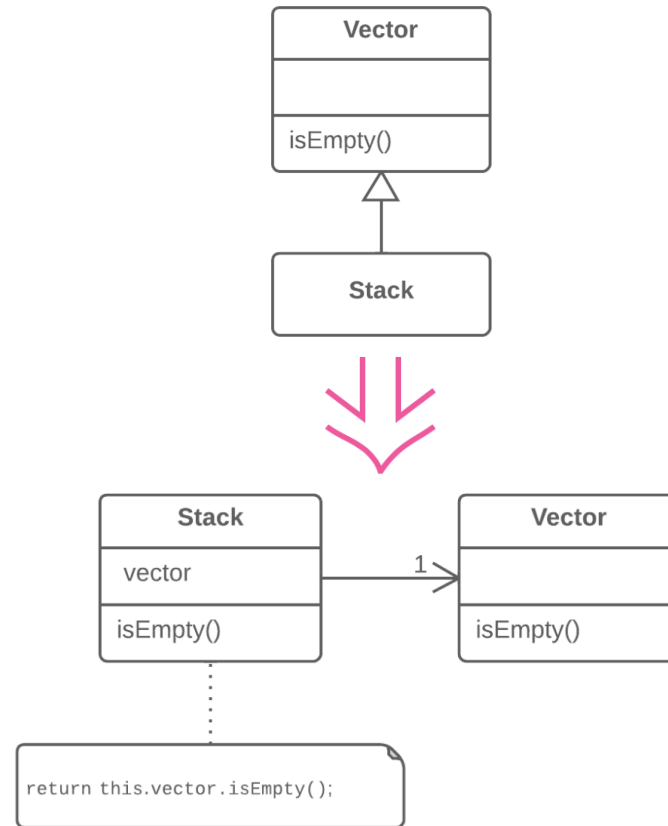
# Refused Bequest

- Sintomo: una sottoclasse utilizza solo alcuni dei metodi e delle proprietà ereditati dai suoi genitori, la gerarchia non è corretta.
- Problema: I metodi non necessari possono semplicemente rimanere inutilizzati o essere ridefiniti e dare luogo a eccezioni. Soluzione:
  - *Se l'ereditarietà non ha senso e la sottoclasse non ha davvero nulla in comune con la superclasse, eliminare l'ereditarietà a favore di **Replace Inheritance with Delegation**.*
  - *Se l'ereditarietà è appropriata, eliminare i campi e i metodi non necessari nella sottoclasse. Estrarre dalla classe padre tutti i campi e i metodi necessari alla sottoclasse, inserirli in una nuova superclasse e impostare entrambe le classi in modo che ereditino da essa (**Extract Superclass**).*



# Replace Inheritance with Delegation

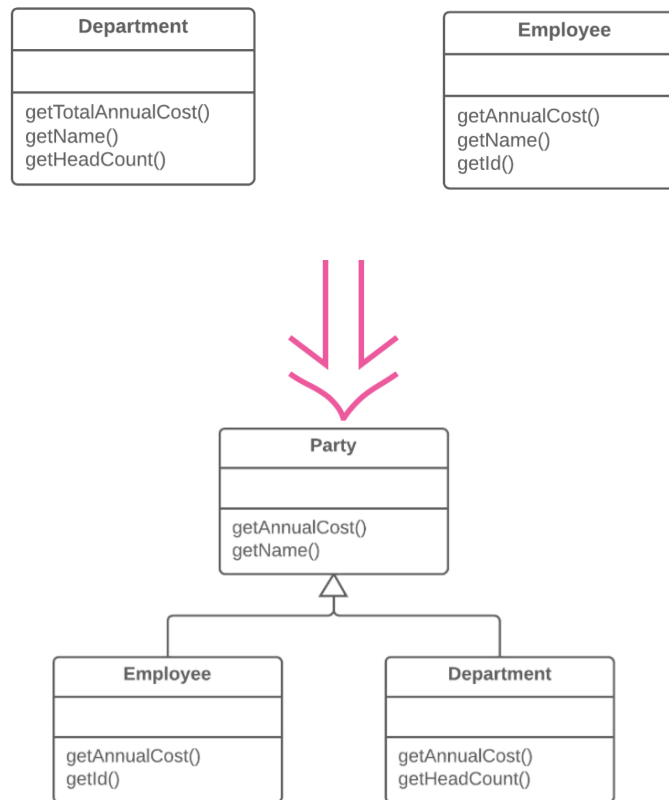
Si ha una sottoclasse che utilizza solo una parte dei metodi della sua superclasse (oppure non è possibile ereditare i dati della superclasse).



**Creare un campo e inserirvi un oggetto della superclasse, delegare i metodi all'oggetto della superclasse e rimuovere l'ereditarietà.**

# Extract Superclass

Si hanno due classi con campi e metodi comuni.



Creare una superclasse condivisa per loro e trasferirvi tutti i campi e i metodi comuni.